

Universidad Nacional de Colombia – Sede Medellín
Facultad de Ciencias – Departamento de Matemáticas

Informe Final: Detección de Fraude Financiero mediante Autoencoders

Trabajo Final – Programación Científica (2026-1)

Mateo Mora — Alejandro Ramírez — Juan Felipe Ramirez
Repositorio del Proyecto: `elliptic-fraud-detection` (branch: `dataset_autoencoder`)
Docente: Manuela Bastidas Olivares

Agosto de 2026

Resumen

El presente informe documenta el desarrollo y evaluación de una arquitectura de **Autoencoder** implementada en PyTorch para la detección de transacciones fraudulentas con tarjetas de crédito. Se aborda el problema del extremo desbalance de clases mediante un enfoque de aprendizaje semisupervisado, donde la red aprende la representación comprimida de transacciones legítimas para identificar anomalías basándose en el error de reconstrucción. Este proyecto integra conceptos de aprendizaje profundo, preprocesamiento de datos, optimización basada en gradientes y visualización científica vistos durante el curso.

1. ¿Por qué eligieron este tema? (Motivación del grupo)

La detección de fraude en transacciones financieras representa uno de los problemas más críticos y de mayor impacto práctico en la intersección entre la ciencia de datos, el aprendizaje automático y las finanzas cuantitativas. Elegimos este tema porque nos permitió conectar directamente los fundamentos teóricos abordados a lo largo del curso de *Programación Científica* —tales como la reducción de dimensionalidad, la aproximación de funciones complejas, la optimización mediante gradiente descendente y la arquitectura de redes neuronales— con una aplicación real de alta relevancia.

Uno de los integrantes del equipo ya contaba con experiencia previa en modelos de aprendizaje profundo y detección de anomalías, lo que sirvió como catalizador para seleccionar la arquitectura de **Autoencoders**. Para los demás integrantes, el proyecto constituyó una valiosa oportunidad de aprendizaje guiado para profundizar en el marco de trabajo PyTorch, estructurar flujos de trabajo en Python orientados a la ciencia de datos y comprender los desafíos asociados al manejo de conjuntos de datos altamente desbalanceados (donde las transacciones fraudulentas representan menos del 0.2% del total de observaciones).

Asimismo, el proyecto permitió integrar y afianzar competencias clave desarrolladas en la asignatura: desde el análisis exploratorio de datos (EDA) y la estandarización de características, hasta el entrenamiento iterativo de modelos probabilísticos/profundos, la selección óptima de umbrales de decisión y la evaluación rigurosa de métricas de desempeño (F_1 -score, PR-AUC y matrices de confusión).

2. ¿Por qué es interesante en la ciencia? (Contexto histórico y relevancia)

Con la masificación del comercio electrónico y la banca digital, el volumen de transacciones financieras a nivel global creció a escalas exponenciales, haciendo inviable la verificación manual o la dependencia exclusiva de sistemas basados en reglas estáticas definidas por expertos.

- **Evolución histórica:** En sus inicios, la detección de anomalías se abordó mediante métodos estadísticos tradicionales (p. ej., distancias de Mahalanobis y modelos de mezcla gaussiana) y, posteriormente, con algoritmos de aprendizaje supervisado convencionales como *Random Forests* y *Support Vector Machines* (SVM). Sin embargo, estos enfoques supervisados presentan dos limitaciones fundamentales en entornos reales:

1. Requieren grandes cantidades de datos previamente etiquetados como fraude, los cuales son escasos y costosos de obtener.
2. Presentan una baja capacidad de generalización ante patrones de fraude emergentes o no registrados históricamente (*zero-day fraud*).

- **Relevancia de los Autoencoders:** Popularizados en la literatura del aprendizaje profundo por investigadores como Geoffrey Hinton y Yoshua Bengio, los **Autoencoders** abordan el problema desde la perspectiva del *aprendizaje de representaciones* y la *detección de anomalías no supervisada/semisupervisada*. La red neuronal se entrena para aprender una proyección comprimida (*manifold* o espacio latente) que captura la estructura esencial de los datos **normales**.

Matemáticamente, la red aprende una función de codificación $g_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^k$ y una función de decodificación $f_\theta : \mathbb{R}^k \rightarrow \mathbb{R}^D$ (con $k \ll D$), minimizando la función de pérdida de reconstrucción sobre el conjunto de datos normales $\mathcal{X}_{\text{normal}}$:

$$\min_{\theta, \phi} \frac{1}{N} \sum_{i=1}^N \|x_i - f_\theta(g_\phi(x_i))\|_2^2 \quad (1)$$

Cuando el modelo procesa una transacción normal, el error de reconstrucción $E_i = \|x_i - \hat{x}_i\|^2$ es cercano a cero. Por el contrario, cuando ingresa una transacción fraudulenta cuyas propiedades difieren de la distribución aprendida, la red no logra reconstruirla adecuadamente, produciendo un error de reconstrucción sensiblemente elevado. Esta propiedad convierte a los autoencoders en un área de investigación sumamente activa en ciberseguridad, bioinformática, mantenimiento predictivo industrial y finanzas.

3. Ejemplo aplicado (Notebook y Modelo)

El desarrollo técnico se estructuró de manera reproducible dentro del repositorio del proyecto [2], utilizando el conjunto de datos *Credit Card Fraud Detection* publicado por el grupo de investigación MLG-ULB en Kaggle [1]. Este conjunto contiene 284,807 transacciones, de las cuales solo 492 son fraudulentas (0.172 %).

3.1. Pipeline de Preprocesamiento y Entrenamiento

1. **Estandarización de Datos:** Las características $V_1, V_2, \dots, V_{28}$ (provenientes de una transformación PCA previa) junto con la variable **Amount** se escalaron mediante **StandardScaler** de Scikit-Learn.
2. **Partición de Datos no Supervisada:** Se filtró el conjunto de entrenamiento para incluir **únicamente transacciones normales** ($y = 0$), reservando una fracción de datos normales y la totalidad de los datos fraudulentos ($y = 1$) para los conjuntos de validación y prueba.
3. **Arquitectura de la Red (PyTorch):** Definida en el módulo `src/model.py`, la red **Autoencoder** consta de una estructura totalmente conexa con capas de normalización por lotes (**BatchNorm1d**), funciones de activación no lineales (**LeakyReLU**) y regularización estocástica (**Dropout**):
 - **Encoder:** Compresión progresiva de entrada $D = 29 \rightarrow 64 \rightarrow 32 \rightarrow k$ (espacio latente).
 - **Decoder:** Reconstrucción del vector de entrada original $k \rightarrow 32 \rightarrow 64 \rightarrow D = 29$.
4. **Optimizador y Criterio:** Se empleó la pérdida de error cuadrático medio (MSE) y el optimizador Adam con tasa de aprendizaje adaptativa.
5. **Inferencia y Detección:** Se fijó un umbral de decisión τ sobre la distribución del error de reconstrucción E_i . Si $E_i > \tau$, la transacción se clasifica como fraude. El umbral óptimo se determinó maximizando la métrica F_1 -score sobre la curva Precision-Recall (PR-AUC).

4. Opinión del grupo y Autoevaluación

4.1. Opinión individual: Alejandro Ramírez

“En mi opinión, el tema elegido fue muy acertado porque permitió aplicar de manera práctica varios de los conceptos vistos durante el curso. La detección de fraude mediante autoencoders es un problema actual y de gran importancia, además de representar un reto interesante debido al desbalance de los datos. A través del proyecto pude comprender mejor el proceso completo de desarrollo de un modelo de aprendizaje automático, desde el análisis y preprocesamiento de los datos hasta la evaluación de su desempeño.

En cuanto al curso, considero que fue una experiencia muy enriquecedora. Los temas de visualización científica, redes neuronales, diferenciación automática con JAX y métodos de aproximación proporcionaron una visión amplia de diferentes herramientas utilizadas en la computación científica. Personalmente, las redes neuronales fueron el tema que más despertó mi interés por sus aplicaciones en problemas reales.

Respecto a mi desempeño, considero que logré comprender los conceptos principales y aplicarlos en los talleres y el proyecto final. Sin embargo, me gustaría seguir fortaleciendo mis conocimientos, especialmente en los fundamentos matemáticos de las redes neuronales y en el uso de herramientas como JAX. En general, considero que el curso me permitió desarrollar nuevas habilidades que serán de gran utilidad en futuros proyectos relacionados con ciencia de datos e inteligencia artificial.”

4.2. Opinión individual: Mateo Mora

“El desarrollo de este proyecto me permitió consolidar el aprendizaje sobre arquitecturas profundas y su implementación limpia en PyTorch, conectando la teoría de optimización y reducción de dimensionalidad con un repositorio estructurado profesionalmente. El curso de Programación Científica brindó una perspectiva rigurosa sobre cómo construir software científico reproducible, eficiente y bien documentado. Valoro enormemente el aprendizaje sobre el flujo de trabajo con Git/GitHub, la diferenciación automática y la visualización científica efectiva. Considero que mi desempeño en el curso y en el proyecto fue muy positivo, logrando aportar en el diseño del modelo y en la estructura modular del código.”

4.3. Opinión individual: Juan Felipe Ramírez

“Considero que abordar la detección de fraude mediante autoencoders fue una elección muy acertada, ya que nos enfrentó a un reto real y complejo del análisis de datos como lo es el severo desbalance de clases. El proyecto me permitió consolidar los conceptos teóricos abordados en la materia, conectando la reducción de dimensionalidad y el entrenamiento de modelos profundos en PyTorch con una evaluación rigurosa del desempeño.

En cuanto a la asignatura, valoro especialmente el énfasis en las buenas prácticas de la computación científica, desde la diferenciación automática hasta la visualización y reproducibilidad del código. Respecto a mi desempeño, considero que logré una comprensión suficiente de las herramientas del curso y una participación activa en el desarrollo del proyecto final, adquiriendo habilidades clave para el análisis de datos e inteligencia artificial.”

Referencias

- [1] Machine Learning Group – ULB, *Credit Card Fraud Detection Dataset*, Kaggle, 2018. URL: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/data>.
- [2] M. Mora, A. Ramírez y J. F. Ramírez, *Elliptic Fraud Detection - Autoencoder Model*, Repositorio en GitHub, 2026. URL: https://github.com/Mmora-07/elliptic-fraud-detection/tree/dataset_autoencoder.